

연구논문/작품 중간보고서

2026 학년도 제 2 학기

제목	GPU 프로파일링·시뮬레이션 기반 LLM 추론 attention 커널 동적 선택 시스템	○ 논문() 작품(✓) ※해당란 체크
GitHub URL	https://github.com/sunjae0224/kernelscope	
평가등급	지도교수 수정보완 사항	팀원 명단
A,B,F 중 택 1 (지도교수가 부여)	○ ○ ○	이 선 재 (인) (학번: 2021313805)

2026 년 9 월 27 일

지도교수 : 민 동 문 서명

■ 요약

본 작품은 길이가 서로 다른 요청을 함께 처리하는 LLM 서빙 환경에서 attention 커널의 KV 분할 수를 동적으로 선택하는 시스템이다. FlashAttention-2 의 기본 휴리스틱은 배치 크기와 최대 KV 길이를 기준으로 병렬성을 판단하므로, 소수의 긴 요청과 다수의 짧은 요청이 섞이면 일부 스레드 블록에 작업이 집중될 수 있다. 이를 커널 실측, CTA 별 작업량 분석, GPU 시뮬레이션과 성능 모델로 분석하였다.

RTX 4090 에서 6,801 개 조건을 측정한 결과, 균일 길이 배치의 선택 손실은 중앙값 0.72%였으나 가변 길이 배치에서는 최악의 경우 연속 KV 커널 기준으로 최적 설정보다 12.53 배 느린 커널이 선택되었다. 요청 길이 분포를 반영한 조희 테이블 정책을 Qwen3-4B 에 적용하여, 32K 토큰 요청 1 개와 512 토큰 요청 31 개가 섞인 조건의 토큰 간 지연(TPOT)을 61.06 ms 에서 33.80 ms 로 줄였다. 기본 설정 대비 1.81 배의 개선이며, 5 회 반복 모두에서 생성 토큰이 동일하였다.

두 모델을 사용한 후속 실험 108 회에서는 성능 모델 정책이 가변 길이 조건의 TPOT 을 1.18~1.29 배 개선하였다. 다만 균일 길이 배치에서는 개선이 없었으며 일부 조건에서는 생성 토큰이 일치하지 않았다. 성능 모델의 최악 선택 손실과 Accel-Sim 의 절대 시간 오차는 개선 과제로 남아 있다. 실측값을 함께 활용하는 혼합 정책은 오프라인 평가에서 손실 기준을 통과했으며, GPU 없이 원본 기록의 수치를 재계산하는 검증 37 개 항목도 모두 통과하였다. 남은 기간에는 혼합 정책의 실제 생성 검증, 출력 불일치 진단 자동화, 시뮬레이터 보정과 기존 방법 비교를 진행한다.

주요어: LLM 추론, 연속 배치, FlashAttention, KV 분할, GPU 성능 분석

■ 1. 서론

가) 제안배경 및 필요성

(1) LLM 서빙과 디코드 attention

Transformer[1] 구조에 기반한 대규모 언어 모델이 검색, 대화, 코드 작성 등 다양한 서비스에 활용되면서, 모델을 학습시키는 비용뿐 아니라 이미 학습된 모델로 답을 생성하는 추론(inference) 비용이 서비스 운영의 핵심 비용이 되었다. LLM의 추론은 입력 문장 전체를 한 번에 처리하는 프리필(fill) 단계와, 이후 토큰을 하나씩 생성하는 디코드(decode) 단계로 나뉜다. 디코드 단계는 생성할 토큰 수만큼 반복되므로 전체 생성 시간의 대부분을 차지하며, 사용자는 이를 토큰 간 지연(TPOT, time per output token)으로 체감한다.

GPU 한 장이 처리할 수 있는 연산량에 비해 요청 하나의 디코드 연산은 매우 작기 때문에, 서빙 시스템은 여러 요청을 하나의 배치로 묶어 처리한다. Orca[2]는 요청 단위가 아니라 디코드 반복(iteration) 단위로 배치를 재구성하는 스케줄링을 제안하였고, vLLM[3]은 요청마다 길이가 다른 KV 캐시를 고정 크기 페이지 단위로 관리하는 PagedAttention을 제안하였다. 이러한 연속 배치 방식에서는 서로 다른 시점에 도착하여 서로 다른 길이의 문맥을 가진 요청들이 같은 배치 안에서 함께 처리된다. 실제 서비스에서는 긴 문서를 요약하는 요청과 짧은 대화 요청이 섞이는 것이 일반적이므로, 한 배치 안의 KV 캐시 길이는 수십 배 이상 차이가 날 수 있다.

디코드 단계의 attention은 새로 생성할 토큰 하나의 질의(query) 벡터가 지금까지 저장된 모든 키(key)와 값(value)을 읽어 가중합을 구하는 연산이다. 한 번 읽은 데이터에 대해 수행하는 연산이 적기 때문에 메모리 대역폭에 의해 성능이 제한되며, 병렬로 처리할 수 있는 작업의 양은 질의 길이가 아니라 배치 크기와 헤드 수에 의해 결정된다. 배치가 작거나 헤드 수가 적으면 GPU의 스트리밍 멀티프로세서(SM)를 모두 채우지 못한다. Flash-Decoding[4]은 이 문제를 해결하기 위해 각 요청의 KV 캐시를 여러 조각으로 나누어 서로 다른 스레드 블록이 병렬로 처리한 뒤 결과를 합치는 KV 분할(split-KV) 방식을 제안하였고, 이 방식은 FlashAttention-2[5] 라이브러리의 디코드 경로에 포함되어 널리 사용되고 있다.

본 작품은 이 디코드 attention을 대상으로 한다. 이미 구성된 배치와 모델 가중치를 유지한 채 커널의 실행 설정만 바꾸므로, 배치 구성이나 메모리 관리 정책과 분리하여 효과를 측정할 수 있다. 관심 지표는 커널 실행 시간뿐 아니라 실제 생성 과정에서 사용자가 체감하는 TPOT이다.

(2) 요청 길이 차이와 커널 선택의 문제

이때 KV 캐시를 몇 조각으로 나눌지(분할 수, num_splits)는 라이브러리가 배치 크기, 헤드 수, 최대 KV 길이만을 보고 휴리스틱으로 정한다. 이 휴리스틱은 배치 안의 모든 요청이 최대 길이만큼의 작업을 가진다고 가정한다. 따라서 실행되는 스레드 블록의 수가 SM 수의 일정 비율을 넘으면 GPU가 이미 충분히 채워졌다고 판단하여 분할을 하지 않는다. 그러나 길이가 섞인 배치에서는 소수의 긴 요청을 담당할 스레드 블록에 작업이 집중되고, 짧은 요청을 담당할 블록은 조기에 종료되어 나머지 SM이 유휴 상태가 되는 부하 불균형(load imbalance)이 발생한다. 이러한 비효율은 평균 지연 시간과 같은 시스템 수준의 지표만으로는 원인을 파악하기 어려우며, 커널 수준의 측정과 GPU 실행 구조에 대한 분석이 함께 필요하다.

또한 한 종류의 GPU 에서 얻은 측정만으로는 SM 수나 메모리 대역폭이 다른 하드웨어에서 같은 현상이 어떻게 나타날지 답할 수 없다. 이를 위해서는 GPGPU-Sim[6], Accel-Sim[7]과 같은 사이클 수준 GPU 시뮬레이터나 실측으로 보정한 성능 모델을 활용하여 하드웨어 조건을 바꾸어 보는 민감도 분석 (sensitivity analysis)이 필요하다. 이상을 정리하면 본 작품의 필요성은 다음과 같다.

- 실제 서빙 조건인 가변 길이 배치에서 라이브러리의 커널 설정 선택이 얼마나 손해를 보는지 정량적으로 측정할 필요가 있다.
- 그 원인을 스레드 블록 구성과 SM 점유율, 메모리 대역폭 등 GPU 실행 구조의 관점에서 설명할 필요가 있다.
- 커널 수준의 개선이 실제 사용자가 체감하는 토큰 간 지연으로 이어지는지, 전체 모델의 생성 과정에서 검증할 필요가 있다.
- 설정 변경이 생성 결과를 바꾸지 않는지, 즉 생성 결과가 동일한지를 성능과 함께 검증할 필요가 있다.

따라서 단순히 가장 빠른 커널을 찾는 것만으로는 충분하지 않다. 동일한 입력에서 후보를 비교하고, 선택에 드는 CPU 시간과 전체 디코드 시간을 함께 기록하며, 생성 결과가 유지되는 조건과 달라지는 조건을 구분해야 한다. 이러한 기준을 측정 도구와 실제 모델 실행 엔진에 공통으로 적용하였다.

나) 연구논문/작품의 목표

(1) 최종 목표와 세부 목표

본 작품의 최종 목표는 요청 길이 분포에 따라 attention 커널의 분할 수를 매 디코드 단계마다 동적으로 선택하는 시스템을 구현하고, 그 효과를 실제 LLM 의 생성 과정에서 검증하는 것이다. 새로운 attention 수식이나 새로운 커널을 만드는 것이 아니라, 수학적으로 같은 결과를 내는 기존 커널의 실행 설정만을 바꾼다는 점이 특징이다. 이 원칙에 따라 성능 개선은 생성 토큰이 기본 설정과 동일한 경우에만 개선으로 인정한다. 세부 목표와 중간 시점의 진행 상황은 표 1 과 같다.

표 1. 세부 목표와 중간 시점 진행 상황

구분	목표	중간 시점 결과
커널 측정	동일 입력으로 분할 수와 KV 형식을 비교	8 개 측정 집합, 6,801 개 조건, 오류 0 건
원인 분석	CTA 작업량과 하드웨어 자원으로 원인을 설명	부하 불균형 확인, Accel-Sim 43 건 재실행
선택 정책	휴리스틱·고정값·테이블·모델 정책 구현	4 종 정책 구현, 모델 첫 선택 비용 약 59 배 단축
실제 모델 검증	TPOT 과 생성 토큰 일치도를 함께 평가	Qwen3-4B 1.81 배 개선, 후속 실험 108 회
재현과 시각화	GPU 없이 결과 재계산 및 탐색	37 개 검증 통과, 대시보드 4 개 화면

(2) 비교 범위와 평가 기준

후보는 동일한 attention 계산을 수행하는 FlashAttention-2 의 실행 변형이다. 연속 KV 캐시와 페이지 단위 KV 캐시는 별도로 비교하며, 캐시를 비운 cold 상태와 재사용하는 warm 상태도 구분한다. 커널 선택 손실은 선택한 실행 시간을 같은 조건의 최적 후보 시간으로 나눈 뒤 1 을 뺀 값으로 정의한다.

실제 모델에서는 같은 가중치·입력·요청 순서와 도착 시점을 사용하고 정책만 교체한다. 기본 휴리스틱과 생성 토큰이 같은 경우를 출력 동일성이 확인된 개선으로 보고, 불일치한 경우에는 성능 수치와 함께 불일치 위치 및 로짓 차이를 분석한다.

다) 연구논문/작품 전체 overview

(1) 시스템 구성

시스템은 커널 측정, 원인 분석, 분할 수 선택, 실제 모델 실행, 결과 검증, 재현과 시각화의 여섯 단계로 구성된다. 입력은 한 배치에 포함된 요청들의 KV 길이 분포이며, 선택 정책은 매 디코드 단계에서 이 정보를 받아 분할 수를 반환한다. 그림 1은 각 단계의 관계를 보여 준다.

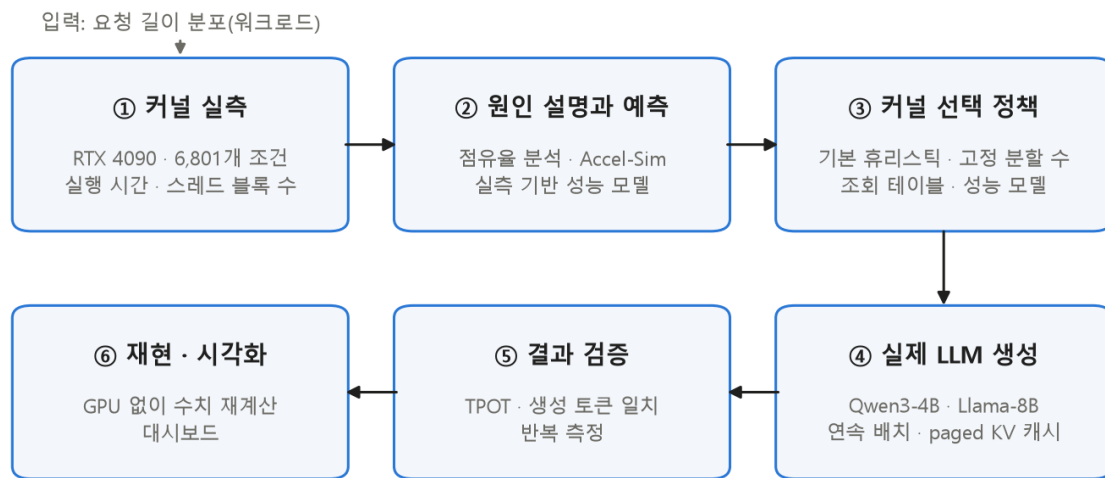


그림 1. 요청 길이 분포에 따른 attention 커널 선택과 검증 과정

커널 실측은 RTX 4090에서 후보별 실행 시간과 CTA 구성을 기록한다. 분석 단계는 CTA 작업량, Accel-Sim과 실측 기반 성능 모델을 이용하여 병목의 원인을 설명한다. 선택 정책은 실측값 또는 예측값을 바탕으로 분할 수를 고르며, 실제 모델 엔진은 Qwen3-4B와 Llama 계열 8B를 연속 배치 및 페이지 단위 KV 캐시로 실행한다.

결과 검증에서는 TPOT과 생성 토큰 일치 여부를 반복 측정한다. 원본 기록은 분석 및 시각화 단계로 전달되므로 GPU를 확보하지 못한 환경에서도 주요 수치를 다시 계산하고 대시보드로 확인할 수 있다. 측정과 재현 단계의 분리는 실험 장비와 보고서 작성 환경이 달라도 같은 근거를 확인하기 위한 설계이다.

(2) 중간 시점의 주요 결과와 보고서 구성

첫째, 균일 길이 배치에서는 기본 휴리스틱의 선택 손실 중앙값이 0.72%로 작았지만, 가변 길이 배치에서는 172.4%로 커졌다. 최악 조건에서는 최적 설정보다 12.53배 느린 커널이 선택되었다. CTA별 작업량 분석은 긴 요청을 담당하는 소수의 블록에 작업이 집중되는 것이 원인임을 보여 주었다.

둘째, 조회 테이블 정책은 Qwen3-4B의 가변 길이 조건에서 TPOT을 1.81배, 실행 도중 요청이 추가되는 조건에서는 1.17배 개선하였다. 두 조건 모두 5회 반복에서 기본 설정과 같은 토큰을 생성하였다. 균일 길이에서는 개선이 없었으며, 이는 휴리스틱이 이미 최적에 가까웠던 커널 측정 결과와 일치한다.

셋째, 두 모델과 직접 작성한 자연어 입력을 사용한 후속 실험 108 회에서 성능 모델 정책은 가변 길이 TPOT 을 Qwen3-4B 에서 1.28~1.29 배, Llama 계열 8B 에서 1.18~1.19 배 개선하였다. 일부 조건은 출력 검증에 실패하였으므로, 이 결과를 모든 입력에 일반화하지 않는다.

넷째, 성능 모델의 시간 예측 오차 중앙값은 2.3~12.3%였지만 최악 선택 손실은 24~66%로 목표에 미달하였다. 이후 실측 테이블을 결합한 혼합 정책은 cold 상태의 오프라인 평가에서 최악 손실 15% 기준을 통과하였다. 실제 생성에서의 검증은 남아 있다. GPU 없이 원본 기록을 재계산하는 검증은 37 개 항목이 모두 통과하였다.

2 장에서는 LLM 서빙, attention 커널 최적화, GPU 성능 분석에 관한 연구와 원안과의 관계를 정리한다. 3 장에서는 이론적 배경, 시스템 구성과 구현, 선택 정책을 설명한다. 4 장에서는 커널 및 실제 모델의 측정 결과, 성능 모델과 시뮬레이터 검증, 한계와 보완 계획을 분석한다. 5 장은 결론과 연구 과정에서 얻은 경험을 정리한다.

■ 2. 관련연구

가) LLM 추론과 서빙 시스템

Transformer[1]는 순환 구조 없이 attention 만으로 시퀀스를 처리하는 구조로, 현재 대부분의 대규모 언어 모델의 기반이 되었다. 자기회귀 방식으로 토큰을 생성할 때에는 이전 토큰들의 키와 값을 KV 캐시에 저장해 두고 재사용한다. 본 작품의 실험에 사용한 Qwen3[8]는 0.6B 부터 235B 규모까지의 dense 및 MoE 모델로 구성된 공개 모델 계열로, 여러 질의 헤드가 하나의 키·값 헤드를 공유하는 grouped-query attention(GQA) 구조를 사용한다.

Orca[2]는 요청 전체가 끝날 때까지 기다리지 않고 디코드 반복 단위로 배치를 다시 구성하는 반복 단위 스케줄링(iteration-level scheduling)을 제안하여, 길이가 서로 다른 생성 요청을 효율적으로 함께 처리할 수 있게 하였다. vLLM[3]은 운영체제의 가상 메모리처럼 KV 캐시를 고정 크기의 블록 단위로 관리하는 PagedAttention 을 제안하여 가변 길이 KV 캐시로 인한 메모리 낭비를 크게 줄였다. Sarathi-Serve[9]는 긴 프리필 요청을 여러 조각으로 나누어 진행 중인 디코드와 함께 처리하는 스케줄링으로 처리량과 지연 시간 사이의 균형을 개선하였다. 이들 연구가 확립한 연속 배치와 페이지 단위 KV 캐시 구조가 곧 본 작품이 다루는 가변 길이 배치가 발생하는 환경이다.

국내 연구에서 김수진·유현창[10]은 질의 간 의미적 유사도로 요청을 군집화하고 KV 캐시를 미리 구축·재사용하는 시스템을 제안하였다. 최우형 외[11]는 GPU 메모리 용량의 한계를 보완하기 위해 GPU·호스트 메모리와 저장장치를 계층적으로 활용하는 사례를 정리하였다. 전자는 요청 사이의 재사용 가능성을, 후자는 KV 데이터가 저장되는 위치를 중심으로 서빙 비용을 줄이는 접근이다.

이들 연구는 본 작품의 실험 환경과 문제 범위를 구분하는 기준이 된다. 본 작품은 캐시에 보존할 데이터나 배치에 포함할 요청을 바꾸지 않고, 이미 구성된 배치의 attention 을 단일 GPU 에서 실행할 때 분할 수를 선택한다. 따라서 요청 스케줄링이나 KV 메모리 관리의 효과와 커널 실행 설정의 효과를 별도로 측정한다.

나) attention 커널 최적화

FlashAttention[12]은 GPU 의 고대역폭 메모리(HBM)와 온칩 SRAM 사이의 데이터 이동을 최소화하는 타일링 기법으로, 근사 없이 정확한 attention 을 빠르고 메모리 효율적으로 계산하였다. FlashAttention-2[5]는 스레드 블록과 워프 사이의 작업 분할 방식을 다시 설계하여 GPU 점유율을 높이고 처리량을 개선하였다. Flash-Decoding[4]은 디코드 단계에서 질의 길이가 1 이기 때문에 배치가 작으면 SM 을 충분히 활용하지 못하는 문제를 지적하고, KV 캐시를 여러 조각으로 나누어 병렬로 계산한 뒤 log-sum-exp 방식으로 부분 결과를 합치는 방법을 제안하였다. 본 작품이 선택하는 대상인 분할 수가 바로 이 방법의 핵심 설정값이다.

FlashDecoding++[13]는 부분 softmax 의 동기화 비용, 작은 행렬 곱의 낮은 활용률, 고정 실행 방식의 비효율을 unified max value, flat GEMM 최적화, 입력·하드웨어별 휴리스틱 선택으로 개선하였다. 입력 형태에 따라 실행 방식을 선택한다는 점은 본 작품과 공통적이다.

FlashInfer[14]는 다양한 KV 형식을 블록 회소 행렬 형식으로 통일하고, 요청별 작업량 차이를 고려하는 부하 분산 스케줄링과 JIT 컴파일 기반 attention 템플릿을 제공한다. 가변 길이 배치의 부하 불균형을 다룬다는

점에서 가장 가까운 연구이다. 다만 FlashInfer가 커널 내부 스케줄링을 조정하는 데 비해, 본 작품은 기존 FlashAttention-2의 분할 수 인자를 외부에서 선택한다. FlashInfer와의 정량 비교는 향후 과제이다.

다) GPU 성능 분석과 시뮬레이션

(1) GPU 성능 분석

루프라인 모델[15]은 연산 강도, 연산 성능, 메모리 대역폭을 함께 보아 병목을 판단한다. 본 작품은 커널의 최소 이동 바이트와 연산량을 계산하고 실측 대역폭 · 연산 성능과 비교한다. 국내의 정종현 외[16]는 마이크로벤치마크로 최신 GPU의 데이터 캐시 동작을 측정하여 계층별 성능 특성을 분석하였다. 이처럼 실행 시간과 내부 자원 특성을 함께 살펴보는 접근을 분할 수 선택 분석에 적용하였다.

(2) GPU 시뮬레이션과 본 작품의 위치

GPGPU-Sim[6]은 PTX 가상 명령어를 실행하는 상세 GPU 시뮬레이터로, 캐시 · 인터커넥트 · 메모리 컨트롤러 등 설계 요소의 영향을 분석한다. Accel-Sim[7]은 실제 GPU에서 수집한 SASS 명령어 추적과 검증 방법론을 제공한다. 본 작품은 Accel-Sim의 RTX 4090 구성에서 SM 수, 메모리 대역폭과 L2 크기를 바꾸어 민감도를 분석하고, GPGPU-Sim에서는 직접 작성한 축소 커널로 부하 불균형의 경향을 확인하였다.

라) 원안과의 관계 및 접근 방법 비교

원안은 Transformer와 선택적 상태 공간 모델 Mamba[17]를 입력에 따라 전환하는 시스템이었다. 그러나 두 모델은 구조와 가중치가 달라 생성 결과 자체가 바뀐다. 이에 전환 대상을 같은 모델 안의 attention 실행 설정으로 좁혔다. 표 2는 관련 연구와 본 작품의 접근을 비교한 것이다.

표 2. 관련 연구와 본 작품의 접근 비교

연구	접근 방법	본 작품과의 관계
Orca[2] · vLLM[3]	반복 단위 스케줄링과 페이지 KV 관리	가변 길이 배치가 발생하는 환경
Sarathi-Serve[9]	프리필 분할과 디코드 동시 처리	배치 구성 수준에서 상호 보완
Flash-Decoding[4] FlashAttention-2[5]	KV 분할과 부분 결과 결합	선택 대상과 기본 휴리스틱
FlashDecoding++[13] FlashInfer[14]	실행 방식 선택과 내부 부하 분산	관련 최적화 및 향후 비교 대상
본 작품	실측 · 모델 기반 분할 수 선택	기존 커널을 유지하며 TPOT · 출력 검증

■ 3. 제안 작품 소개

가) 이론적 배경

(1) 디코드 attention 과 메모리 병목

요청 b 의 헤드는 질의 q 와 길이 L_b 인 키 행렬 K , 값 행렬 V 로 $o = \text{softmax}(qK^T/\sqrt{d})V$ 를 계산한다. Qwen3-4B 는 질의 헤드 32 개가 KV 헤드 8 개를 공유하는 GQA 구조이며 헤드 차원 d 는 128 이다. 한 단계에서 읽는 KV 양은 요청 길이의 합에 비례한다. 디코드에서는 읽는 데이터에 비해 연산량이 작아 메모리 대역폭과 병렬 작업 수가 실행 시간을 좌우한다.

(2) GPU 실행 구조와 부하 불균형

CUDA 커널은 여러 스레드 블록(CTA)의 그리드로 실행된다. 각 CTA 는 한 SM 에 배치되며, 동시에 상주할 수 있는 CTA 수는 레지스터와 공유 메모리 사용량의 제한을 받는다. 점유율(occupancy)은 SM 의 최대 수용 워프 수 대비 활성 워프 수의 비율이다. RTX 4090 은 SM 128 개를 가진다. CTA 별 작업량이 다르면 짧은 작업이 먼저 종료되어 일부 SM 만 동작하는 꼬리 구간이 발생한다.

(3) KV 분할의 이득과 비용

KV 분할은 각 요청·KV 헤드의 캐시를 여러 구간으로 나누어 CTA 들이 부분 attention 을 계산하고, 별도의 결합 커널이 부분 결과를 합치는 방식이다. 본 실험 구성에서 분할 수 S 에 따른 CTA 수는 $S \times B \times H_{kv}$ 이다. 분할은 병렬성을 늘리지만 부분 결과를 저장·재읽기하는 트래픽과 결합 비용도 추가한다. 따라서 분할 수가 지나치게 작으면 SM 을 충분히 활용하지 못하고, 지나치게 크면 부가 비용이 증가한다.

attention 이 디코드 단계에서 차지하는 비중을 f , attention 속도 향상을 s 라 하면, 다른 연산 시간이 같을 때 단계 전체의 속도 향상은 $1/((1-f)+f/s)$ 이다. 실제 모델 결과에서는 attention 시간 변화로 예측한 단계 시간과 측정값을 비교하여 개선의 원인을 확인한다.

나) 기본 휴리스틱과 가변 길이 배치

FlashAttention 2.8.3 은 기본 CTA 수 $B \cdot H_{kv} \cdot m$ 이 SM 당 CTA 2 개를 가정한 실행 슬롯 수의 80% 이상이면 분할하지 않는다. 그보다 작으면 1~128 범위에서 계산한 최고 활용 효율의 85% 이상을 얻는 가장 작은 분할 수를 선택한다. m 은 질의 행 블록 수이다. 이 규칙과 그리드 구성을 라이브러리 소스 코드에서 이식하여 CTA 별 작업량을 계산하였다(부록 A.1).

32K 요청 1 개와 512 토큰 요청 31 개에서는 기본 CTA 가 $32 \times 8 = 256$ 개이므로 기준 204.8 을 넘어 분할 수 1 을 고른다. 그러나 32K 요청을 맡은 8 개 CTA 에 긴 작업이 집중되고 나머지 248 개는 조기에 종료된다. 분할 수 16 에서는 긴 요청의 작업이 128 개 CTA 로 나뉘고 CTA 당 최대 키 수가 2,048 개로 줄어든다.

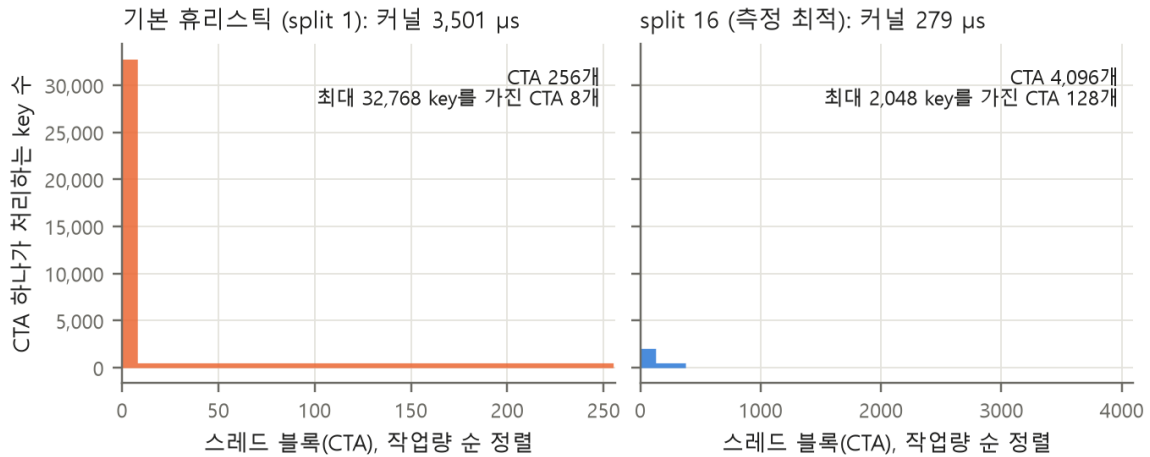


그림 2. 가변 길이 배치에서 분할 수에 따른 CTA 별 키 수

그림 2는 동일한 요청 구성의 이론적 작업량을 내림차순으로 배열한 것이다. 분할 수 16에서는 짧은 요청의 대부분 구간이 비어 있으므로 키를 처리하지 않는 CTA도 포함된다. 즉 CTA의 총개수를 늘리는 것이 아니라, 긴 요청의 작업을 여러 SM에 분산하는 것이 개선의 요인이다. 이 조건의 연속 KV 커널 실행 시간은 3,501 μs에서 279 μs로 줄었다.

선택 손실은 (선택 커널 시간/동일 조건의 최적 후보 시간)-1로 정의한다. 비교 후보와 KV 형식, cold·warm 상태를 고정하여 입력이나 캐시 상태 차이가 선택 정책의 효과로 섞이지 않도록 하였다.

다) 시스템 구성과 구현

시스템은 Python 패키지 약 7,200 줄과 대시보드·스크립트 약 1,700 줄로 구현하였다. 구성 요소는 Parquet·CSV·JSON 결과를 주고받아 GPU 측정과 CPU 분석·재현을 분리한다.

(1) 커널 측정과 실행 구성 분석

분할 수 1, 기본 휴리스틱(num_splits=0), 고정 분할 수 2~128의 7종 변형을 등록하여 조건마다 9개 후보를 비교한다. 연속 KV와 페이지 KV를 구분하고, CUDA 이벤트와 torch.profiler의 CUPTI activity로 시간·그리드·블록·레지스터·공유 메모리를 기록한다. cold 측정에서는 L2 용량의 4배 데이터를 쓰는 커널을 먼저 실행한다. 참조 구현과 결과를 비교하고 GPU 사용률·다른 프로세스도 기록한다.

실행 구성 분석은 CTA별 키 수와 이동 바이트·연산량을 계산한다. Accel-Sim은 NVBit로 수집한 실제 커널 추적을 RTX 4090 구성에서 재실행하고, SM 수·대역폭·L2를 각각 두 배와 절반으로 바꾸어 자원별 민감도를 분석한다. Nsight Compute를 사용할 수 없는 공용 서버 환경을 고려하여 하드웨어 카운터에 의존하지 않도록 하였다.

(2) 실측 기반 성능 모델

성능 모델은 CTA별 키 수를 입력받아 SM 수와 상주 CTA 수로 정한 실행 슬롯에 작업을 배치한다. 동시 실행 CTA가 대역폭을 나누어 쓰는 이산 사건 시뮬레이션으로 완료 시간을 계산하며, 키당 처리 시간·시작 비용·결합 비용은 균일 길이 실측에 Nelder-Mead 방법으로 맞춘다. NumPy 구현을 C언어로 재구현하여 첫 선택 비용을 842.6 ms에서 14.3 ms로 줄였고, 기존 7개 조건에서 예측값과 순위의 일치를 확인하였다.

(3) 재현성 검증과 대시보드

원본 기록과 해시를 결과 아카이브에 저장하고 kernelscope verify 로 문서의 주요 수치를 다시 계산한다. 대시보드는 병목 진단, 최적 커널, 하드웨어 조건 변경, 실제 생성 결과의 네 화면을 제공하며 실측과 예측을 구분한다. 두 기능 모두 GPU 없이 실행된다.

라) 선택 정책과 실제 모델 실행

정책은 매 단계 요청 길이 목록을 받아 분할 수를 반환한다. 구현한 정책은 표 3 과 같다. 테이블·모델 정책은 256 토큰 페이지 단위로 양자화한 길이 목록을 캐시 키로 사용하고, 정책 계산 CPU 시간을 별도로 기록한다. 혼합 정책의 검증 결과는 4 장 라)에서 설명한다.

표 3. 구현한 분할 수 선택 정책

정책	선택 방법
기본 휴리스틱	num_splits=0 으로 라이브러리 규칙 사용
고정값	항상 같은 분할 수 사용(예: 8)
조희 테이블	같은 헤드 구성에서 배치 크기·최대 길이·평균 길이의 로그 거리가 가장 가까운 실측 조건의 최적값 사용
성능 모델	후보 9 개의 실행 시간을 예측해 최소값 선택

실행 엔진은 safetensors 의 bf16 가중치를 읽고 페이지 KV 를 관리하며, flash_attn_with_kvcache 에 선택한 분할 수를 전달한다. 시나리오의 논리적 도착 단계에 따라 요청을 배치에 추가하고 탐욕적 디코딩으로 고정 길이를 생성한다. attention·전체 단계 시간과 토큰 생성 시각을 기록하고, 엔진의 수치적 정확성은 독립된 Hugging Face transformers 구현과 비교하였다.

■ 4. 구현 및 결과분석

가) 실험 환경과 구현 현황

실험 환경은 표 4 와 같다. 8 개 측정 집합에서 워크로드, 캐시 상태, 후보 커널의 조합 6,801 개를 측정하였으며 오류는 없었다. GPU 가 없는 노트북 환경에서는 테스트 470 개가 통과하였고 GPU 전용 12 개와 선택 의존성 관련 9 개는 제외·생략하였다(9 월 25 일). 별도로 서버의 디코더·KV 캐시 GPU 테스트 7 개가 통과하였다.

표 4. 실험 환경

항목	내용
GPU	RTX 4090, SM 128 개, L2 72 MiB 실측 대역폭 952.6 GB/s, 텐서 코어 168.6 TFLOPS
소프트웨어	PyTorch 2.8.0(CUDA 12.8 빌드), CUDA 12.9 FlashAttention 2.8.3.post1, 드라이버 580.95.05
모델·시뮬레이터	Qwen3-4B-Instruct-2507, DeepSeek-R1-Distill-Llama-8B Accel-Sim v2.0.0, NVBit 1.8, GPGPU-Sim 4.2
측정 범위	균일: B=1~64, L=512~32K 가변: B=16~64, 긴 요청 1·2·4 개, 긴 길이 8K·16K·32K, 짧은 길이 512·1K·2K, 헤드 구성 2 종

나) 커널 수준 측정 결과

표 5 는 기본 휴리스틱의 선택 손실을 요약한 것으로, cold 상태의 결과이며 기본 헤드 구성 S1 은 질의 32 개·KV 8 개이다. 균일 길이에서는 휴리스틱이 최적에 가까웠지만 가변 길이에서는 손실이 크게 증가했다. 최악 연속 KV 조건($32K \times 1 + 512 \times 31$)은 12.53 배 느렸다. 같은 입력의 페이지 KV 는 3.84 배로 차이가 작았으며, 실제 엔진이 페이지 KV 를 사용하므로 전체 모델의 개선은 이 경로를 기준으로 해석한다.

표 5. 기본 휴리스틱의 선택 손실 요약

측정 집합	조건 수	중앙값	최댓값	10% 초과
균일·연속 KV	49	0.7%	5.7%	0
균일·페이지 KV	49	1.2%	10.0%	0
가변·연속 KV	162	172.4%	1152.7%	124
가변·페이지 KV	162	43.6%	298.1%	127
가변·페이지 KV(S2)	135	47.2%	432.3%	102

다) 실제 LLM 생성 결과

(1) TPOT 과 출력 동일성

Qwen3-4B 에서 균일 (512×32), 가변 ($32K \times 1 + 512 \times 31$), 요청 도착 시나리오를 비교하였다. 앞의 두 조건은 요청당 64 토큰을 생성한다. 요청 도착은 $16K \times 1 + 512 \times 15$ 로 시작해 8·24 단계에 짧은 요청 16 개씩을 추가한다. 정책별로 실행 순서를 바꾸어 5 회 반복하였다. 결과는 표 6 과 같다.

표 6. Qwen3-4B 의 TPOT 과 출력 동일성

시나리오	기본 TPOT	테이블 TPOT	개선	토큰
균일 길이	27.42 ms	27.50 ms	1.00 배	일치
가변 길이	61.06 ms	33.80 ms	1.81 배	일치
요청 도착	59.09 ms	50.34 ms	1.17 배	일치

가변 길이에서 테이블 정책은 단계당 attention 시간을 36.87 ms 에서 9.24 ms 로 줄였다. 전체 단계에서 attention 비중은 약 72%에서 38%로 낮아졌다. attention 시간 차이만 반영한 예측 단계 시간과 실제 측정값의 차이는 1.5%로, 개선이 attention 커널 선택에 기인함을 확인하였다. 요청 도착에서는 긴 요청 비중이 작아 개선 폭이 1.17 배였다.

균일 길이는 기본 휴리스틱이 이미 최적에 가까워 테이블 정책의 이득이 없었다(0.997 배). 고정 분할 수 8 은 가변 길이에서 TPOT 33.39 ms 를 얻었지만 균일 길이에서는 토큰 일치율이 94.43%에 그쳤다. 분할에 따른 결합 순서와 bf16 반올림 차이가 생성에 영향을 줄 수 있으므로 이를 출력 동일성이 보장된 개선으로 보지 않았다.

초기 NumPy 성능 모델은 가변 길이에서 attention 을 9.13 ms 로 줄였으나 TPOT 은 45.33 ms 였다. 첫 선택의 계산 비용으로 인해 균일 길이와 요청 도착 조건에서는 TPOT 이 오히려 증가하였다. 이에 따라 모델 계산을 C 언어로 재구현하여 첫 선택 비용을 약 59 배 줄였다.

후속 실험은 두 모델 × 자연어 입력 조건 3 개 × 시드 2 개 × 정책 3 개 × 반복 3 회, 총 108 회이다. 가변 길이 ($12K \times 1 + 384 \times 27$) 에서 성능 모델은 Qwen3-4B 의 TPOT 을 1.282~1.290 배, Llama 계열 8B 를 1.184~1.188 배 개선했고 토큰도 같았다. 균일 길이는 0.98~1.00 배로 이득이 없었다.

(2) 출력 불일치 분석

후속 실험의 요청 도착 비교 전부와 가변 길이의 테이블 비교 2 건은 출력이 달라, 비기본 정책 비교 24 건 중 14 건만 토큰 동일성 검증을 통과하였다. 불일치한 조건은 반복마다 같은 위치에서 차이가 시작되었다. 입력을 고정한 teacher forcing 진단에서는 정책별 868 회 중 866 회의 최대 확률 토큰이 같았다.

진단에서 토큰이 뒤바뀐 위치의 상위 두 로짓 차이는 0.125 또는 0.25 였으며 bf16 의 해당 값 범위에서 표현 간격의 1~2 배였다. 이는 확률이 비슷한 후보 사이에서 반올림 차이가 생성 경로를 바꾼다는 설명을 뒷받침한다. 다만 이 진단은 자유 생성 결과에 대한 출력 검증을 대체하지 않는다. 토큰 일치 여부와 함께 불일치 시작 위치 · 로짓 차이를 보고하고, 새 실험에서 진단을 자동화할 계획이다.

라) 성능 모델 검증과 혼합 정책

성능 모델과 혼합 정책의 검증 결과는 표 7 과 같다. V1 은 균일 길이 보간, V5 는 미학습 헤드 구성, V6 는 가변 길이 검증 집합이다. 시간 예측은 각 집합의 목표를 통과했고 선택 손실 중앙값도 0.0%였으나, 최악 손실은 기준 15%를 넘었다. 예측 오차가 작더라도 후보 간 순위가 역전될 수 있기 때문이다.

표 7. 성능 모델과 혼합 정책의 검증

집합·캐시	조건 수	예측 오차 중앙값	모델 최악 선택 손실	혼합 최악 선택 손실
V1·cold	594	9.0%	24.3%	1.97%
V1·warm	594	2.3%	41.7%	41.7%
V5·cold	882	12.3%	66.3%	7.54%
V6·cold	1,971	11.1%	24.5%	10.90%

혼합 정책은 가까운 실측 조건의 최적 변형을 모델의 최선 예측 시간보다 20% 이내로 예측하면 그 변형을 선택한다. 각 조건을 제외하고 테이블을 만드는 leave-one-out 평가에서 cold 집합의 최악 손실이 모두 15% 이내였다. warm 상태의 V1 에서는 혼합 정책을 포함한 모든 정책의 최악 손실이 41.7%로 기준을 충족하지 못하였다. 서빙 엔진은 cold 상태를 가정하므로 본 결과의 결론에는 영향이 없으나, 성능 모델의 warm 대역폭 향은 개선 과제로 남는다. 표 7 의 혼합 결과는 저장된 실측값에 대한 오프라인 평가이며 실제 모델 생성 검증은 아직 수행하지 않았다.

마) 시뮬레이터와 재현성 검증

Accel-Sim 은 FlashAttention-2 의 B=1 조건에서 실측 시간의 1.04~1.06 배였으나, Flash-Decoding 과 큰 배치에서는 1.5~3.1 배 차이를 보였다. L2 파티션 해시 설정에서 48 개 슬라이스 중 16 개에 읽기 요청이 약 2 배 집중되는 편향을 확인하였다. 현재는 절대 시간보다 자원별 민감도 분석에 사용한다. B=16 에서 대역폭을 절반으로 줄이면 시간이 약 97~102% 늘어나는 경향을 확인하였다.

GPGPU-Sim 의 SM7_QV100 구성에서는 직접 작성한 fp32 축소 커널로 기전을 검증하였다. 가변 길이 배치는 분할 수 1 에서 균일 배치보다 8.5 배 오래 걸렸고, 분할 수 16 에서 7.23 배 빨라졌다. 균일 배치는 8 에서 2.25 배로 포화한 뒤 결합 비용으로 느려졌다. 이는 실측과 같은 경향이지만 실제 FlashAttention-2 의 절대 시간이나 최적 분할 수를 재현한 결과는 아니다.

GPU 가 없는 노트북 환경에서 커널·생성·후속 실험·선택 비용·성능 모델의 주요 수치 34 개와 일관성 검사 3 개를 원본 기록으로 재계산했고 모두 일치하였다. 검증에는 약 12 초가 소요되었으며 데이터와 문서의 값이 달라지는지도 확인한다. CPU 시연은 스케줄링·토큰 비교 로직을 검증할 수 있으나 분할에 따른 GPU 성능 차이는 재현하지 않는다.

바) 한계와 보완 계획

실험은 RTX 4090 한 종류와 통제된 입력에 한정되며 실제 서비스의 요청 분포, FlashInfer와의 비교, vLLM 등 운영 서빙 엔진에서의 재현, HTTP 서빙·CUDA Graph·다중 GPU 환경은 검증하지 않았다. 출력 불일치와 시뮬레이터 보정도 남아 있다. 남은 기간의 보완 계획은 표 8과 같다.

표 8. 남은 기간의 보완 계획

항목	후속 작업	시기
출력 검증	불일치 시작 위치와 로짓 차이 진단 자동화	GPU 확보 시
선택 정책	혼합 정책의 실제 생성 성능·출력 검증	GPU 확보 시
시뮬레이터	Accel-Sim L2 파티션 해시 변경 후 재검증	GPU 확보 시
기존 방법 비교	FlashInfer와 커널 시간·TPOT 비교	11 월
운영 엔진 재현	vLLM의 FlashAttention 백엔드에서 동일 조건의 손실 재현	11 월
최종 정리	재현성 검증 화면, 최종 보고서·시연 준비	12 월

■ 5. 결론 및 소감

가) 결론

본 작품은 가변 길이 LLM 배치에서 기본 분할 수 휴리스틱이 소수의 CTA 에 작업을 집중시켜 GPU 를 충분히 활용하지 못하는 문제를 확인하였다. RTX 4090 의 6,801 개 조건 측정과 CTA 작업량 분석을 바탕으로 선택 정책을 구현했으며, Qwen3-4B 의 가변 길이 조건에서 생성 토큰을 유지하면서 TPOT 을 61.06 ms 에서 33.80 ms 로 줄였다. 개선이 없는 조건과 출력 불일치도 함께 확인하였다.

성능 모델은 대부분의 조건에서 적절한 커널을 선택했으나 최악 손실 기준에는 미달하였다. 혼합 정책은 오프라인 평가에서 이를 개선했으며 실제 생성 검증이 남아 있다. 앞으로 출력 진단 자동화, 시뮬레이터 보정과 기존 방법 비교를 통해 적용 범위를 확인할 계획이다.

나) 소감

가장 크게 배운 점은 문제를 정확히 정의하는 일이었다. 처음에는 Transformer 와 Mamba 를 상황에 따라 바꾸려 했지만 두 모델은 서로 다른 답을 낸다. 같은 입력의 출력을 유지하면서 실행을 개선한다는 기준으로 돌아가 전환 대상을 커널 설정으로 좁혔고, 이 기준이 이후 실험의 방향을 정했다.

측정 과정에서는 다른 작업이 실행 중이던 공용 GPU 에서 측정한 기록에서 커널 시간이 전체 지연보다 세 배 긴 값을 발견하였다. 해당 결과를 폐기하고 재측정한 뒤 GPU 사용률과 다른 프로세스 기록을 남기도록 바꾸었다. 도구가 출력한 숫자라도 실행 환경과 일관성을 확인해야 한다는 점을 배웠다.

균일 길이에서 이득이 없거나 성능 모델이 오히려 느려진 결과는 처음에는 아쉬웠지만, 적용 조건을 구분하고 C 구현으로 선택 비용을 줄이는 근거가 되었다. GPU 없는 환경에서 검증 기능을 만들며, 결과를 얻는 것뿐 아니라 다른 사람이 같은 기록에서 수치를 확인할 수 있게 만드는 일도 연구의 일부임을 배웠다.

■ 참고문헌

- [1] A. Vaswani et al., "Attention is all you need," NeurIPS, vol. 30, 2017.
- [2] G.-I. Yu et al., "Orca: A distributed serving system for Transformer-based generative models," OSDI, pp. 521–538, 2022.
- [3] W. Kwon et al., "Efficient memory management for large language model serving with PagedAttention," SOSP, pp. 611–626, 2023.
- [4] T. Dao et al., "Flash-Decoding for long-context inference," PyTorch Blog, 2023.
<https://pytorch.org/blog/flash-decoding/>
- [5] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," ICLR, 2024.
- [6] A. Bakhoda et al., "Analyzing CUDA workloads using a detailed GPU simulator," ISPASS, pp. 163–174, 2009.
- [7] M. Khairy et al., "Accel-Sim: An extensible simulation framework for validated GPU modeling," ISCA, pp. 473–486, 2020.
- [8] A. Yang et al., "Qwen3 technical report," arXiv:2505.09388, 2025.
- [9] A. Agrawal et al., "Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve," OSDI, 2024.
- [10] 김수진·유현창, 「LLM 추론 성능 최적화를 위한 클러스터링 기반 KV 캐시 재사용 시스템」, 한국정보처리학회 추계학술발표대회, pp. 592–595, 2025.
- [11] 최우형 외, 「거대 언어 모델을 서빙하기 위한 GPU 메모리 계층화 사례 연구」, 정보과학회지, 43(4), pp. 37–40, 2025.
- [12] T. Dao et al., "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," NeurIPS, vol. 35, 2022.
- [13] K. Hong et al., "FlashDecoding++: Faster large language model inference with asynchronization, flat GEMM optimization, and heuristics," MLSys, 2024.
- [14] Z. Ye et al., "FlashInfer: Efficient and customizable attention engine for LLM inference serving," MLSys, 2025.
- [15] S. Williams et al., "Roofline: An insightful visual performance model for multicore architectures," Commun. ACM, 52(4), pp. 65–76, 2009.
- [16] 정종현 외, 「최신 GPU 아키텍처의 데이터 캐시 성능 분석」, 한국소프트웨어종합학술대회, pp. 1052–1054, 2021.
- [17] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," COLM, 2024.

■ 부록 A. 원천코드 주요 부분 발췌

전체 코드: <https://github.com/sunjae0224/kernelscope>

A.1 분할 수 휴리스틱 `kernelscope/model/geometry.py`

```
def num_splits_heuristic(batch_nheads_mblocks, num_sms, num_n_blocks, max_splits=128):
    """Exact port of flash-attn 2.8.3 num_splits_heuristic (flash_api.cpp)."""
    if batch_nheads_mblocks >= 0.8 * num_sms:
        return 1
    max_splits = min(max_splits, num_sms, num_n_blocks)
    eff = [0.0] * (max_splits + 1)
    best = 0.0
    for s in range(1, max_splits + 1):
        if s > 1 and math.ceil(num_n_blocks / s) == math.ceil(num_n_blocks / (s - 1)):
            continue
        n_waves = batch_nheads_mblocks * s / num_sms
        eff[s] = n_waves / math.ceil(n_waves)
        best = max(best, eff[s])
    for s in range(1, max_splits + 1):
        if ((s == 1 or math.ceil(num_n_blocks / s) !=
            math.ceil(num_n_blocks / (s - 1)))
            and eff[s] >= 0.85 * best):
            return s
    return 1
```

A.2 조회 테이블 선택 `kernelscope/serve/dispatch.py`

```
class TablePolicy:
    @staticmethod
    def _features(lens):
        return np.log2([len(lens), max(lens), float(np.mean(lens))])

    def choose(self, lens, n_heads, n_kv_heads) -> int:
        _validate(lens, n_heads, n_kv_heads)
        key = (n_heads, n_kv_heads, tuple(math.ceil(n / PAGE) for n in lens))
        self.last_cache_hit = key in self._cache
        if self.last_cache_hit:
            value, self.last = self._cache[key]
            return value
        eligible = self._groups.get((n_heads, n_kv_heads), [])
        if not eligible:
            raise ValueError(
                f"dispatch table has no measured shape with Hq={n_heads}, "
                f"Hkv={n_kv_heads}")
        query = self._features(lens)
        distances = np.square(
            self._features_by_shape[(n_heads, n_kv_heads)] - query
        ).sum(axis=1)
        index = int(np.argmin(distances))
        workload, value = eligible[index]
        self.last = {"workload_key": workload.key(),
                     "distance": float(distances[index] ** 0.5)}
        self._cache[key] = (value, self.last)
        return value
```